Lessons Learned — CS6650 Final Project

Rahul Chinya Jagadeesha

Spring 2026

Where I started

In my midterm mastery I described what I was taking out of the course up to that point as "one engineering loop: design, deploy, measure, adapt, and verify." Homework 6 was the concrete version of that loop for me — I had watched an ECS autoscaling policy move the desired/running task count through 2/2/0, 4/2/2, 4/3/1, 4/4/0 in real time while Locust pushed 1,450 RPS through an ALB. Stopping one ECS task mid-load and watching traffic continue felt like the core resilience lesson of the course. I came out of the midterm confident in the scaling and deployment half of systems thinking: Terraform, ALB, autoscaling policies, CloudWatch metrics, target-group health — that whole stack.

Going into the final project, I thought the hard work would be repeating that loop at bigger scale. It turned out the final project added a whole new axis that HW6 never really stressed: **correctness under concurrency and partial failure**. HW6's service was read-mostly — 100 product-lookup calls per request, nothing that could corrupt the database if a worker crashed. The final project's service moves money. A crash at the wrong moment could duplicate a debit, swallow a credit, or leave accounts permanently inconsistent. That shift — from "keep the service up" to "keep the data right even when the service goes down" — is where almost every new lesson lived.

From "scale the service" to "design for correctness"

The items on my ownership list forced me to think about correctness before performance. Designing the DynamoDB schema, building the idempotency layer, writing the Terraform, implementing pessimistic locking, writing the crash-recovery experiment, and owning `verify_balances.py` — in every single one of those, the question "what is correct" came before "how fast is this." HW6 had trained me to think throughput-first. The final project retrained me to think invariant-first: *what is the property this system must never violate, and what mechanism enforces it?*

For our system, the invariant was stated as a single sentence at the beginning of Week 1: *the sum of balances across all accounts must never change except in response to a transfer, and every transfer must either apply fully or not at all.* Every design decision I made after that, I could trace back to that sentence.

Designing the schema was designing the correctness story

The `accounts` table got a `version` attribute from the very first schema draft. That one integer field is the entire optimistic concurrency story: conditional writes check it, successful writes increment it, and conflicting writers hit `ConditionalCheckFailedException` and retry. The `transactions` table got a `status` field with a small state machine (PENDING → COMPLETED or FAILED) that plays exactly the same role for idempotency: a redelivered message's commit condition includes `status = PENDING`, so if some other worker already marked it COMPLETED, the second commit cannot double-apply. The `account_locks` table got an `expires_at` (TTL lease) from day one because without it, a crashed worker holding a pessimistic lock could deadlock the whole system.

None of those fields existed for performance reasons. They existed because the invariant required them. That was the first time in the course I had designed a schema where every column's purpose was "this field exists to make the invariant hold," not "this field exists because the feature needs it." Parnas' information-hiding principle from the midterm took on a new meaning: the schema is the most stable interface in the system, and volatile decisions (how we implement retries, how many workers we run, whether we use optimistic or pessimistic) have to be kept *out* of the schema so they can change without breaking the invariant.

Writing the idempotency layer as five independent mechanisms

In the abstract, idempotency means "doing it twice has the same effect as doing it once." In practice, the idempotency layer in our system is five separate mechanisms that only work because they compose:

1. The API's `GetTransaction` probe short-circuits an already-known transaction ID.
2. `PutTransactionIfNotExists` uses a conditional write with `attribute_not_exists(transaction_id)` so that even if two API tasks race the same request, only one can create the record.
3. The worker's `Process` function reads the transaction at the top of the loop and short-circuits on `COMPLETED` or `FAILED`, so SQS redelivery is safe.
4. The commit itself is one atomic `TransactWriteItems` with `ConditionExpression: status = PENDING` on the status update. If a race somehow still occurs, the second writer's conditional check fails and nothing is applied.
5. `MarkTransactionFailedIfPending` has `status = PENDING` as its own condition, so the error path cannot overwrite a terminal state either.

Each of those is small. But remove any one of them and there is a scenario (hostile retry pattern, long visibility timeout, worker restart mid-commit) where money gets moved twice. Writing all five and seeing how they compose was the moment I truly understood what the course meant by "idempotency is a property, not a feature."

Implementing pessimistic locking: where I learned about deadlocks

Pessimistic locking was my deepest-dive technical contribution. The skeleton was simple — a lock row in `account_locks` with a TTL lease — but making it correct took two rewrites.

The first rewrite was about deadlock. The obvious implementation takes the sender's lock first, then the receiver's lock. If two transfers cross (A→B and B→A) they will deadlock on each other's first lock forever. The fix is to acquire locks in a total order on `account_id` regardless of direction — if the transfer is A→B we lock A then B, but if the transfer is B→A we *also* lock A then B, because A sorts first. Both transfers take the same locks in the same order, so they serialize instead of deadlocking. I had read this pattern in operating-systems courses but had never had to actually apply it. In the final project it was the difference between "pessimistic mode works" and "pessimistic mode eventually freezes the worker fleet."

The second rewrite was about cleanup. Early versions of my code released locks in a separate `DeleteItem` call *after* the `TransactWriteItems` commit. That created a window where a worker could crash between "commit succeeded" and "locks released," leaving the accounts locked until the TTL expired. The fix was to bundle the lock releases into the same `TransactWriteItems` as the commit. Now the commit and the lock release either both happen or neither happens. A crash

between them is no longer possible — there is no "between them." That was the moment I internalized what atomic transactions actually buy you: they eliminate entire classes of intermediate states that you otherwise have to reason about.

Experiment 2 — the crash-recovery test that made it all real

I owned the worker crash-recovery experiment end-to-end, and it turned out to be the single most satisfying result I got this semester. The design was deceptively simple: submit a transfer with an artificial `PRE_COMMIT_DELAY_MS` of 8 seconds so the worker sits holding the message for a predictable window, then call `aws ecs stop-task` on the running worker 4 seconds in, and watch what happens next.

What happens next is: ECS notices the task is dead and launches a replacement; the SQS visibility timeout on the in-flight message expires and the message becomes re-deliverable; the new worker picks it up, sees `status = PENDING`, runs the same `TransactWriteItems`, and the transfer reaches `COMPLETED`. Total balance across all 101 accounts remained exactly $1,010,000.00 with a difference of $0.00.

The reason this was so satisfying is that every individual piece had to do its job: ECS's task replacement, SQS's visibility-timeout redelivery, the worker's terminal-state short-circuit on redelivery, and the atomic commit that made the commit either fully happen or not happen at all. All four mechanisms composed without me having to coordinate them explicitly. That is what the course meant by "design for failure": you do not prevent the crash, you design the system so the crash is uneventful.

If I were running the experiment again I would automate the crash at multiple points, not just one. The current test only kills the worker during the pre-commit delay; a more thorough test would also kill it between `GetAccount` and `TransactWriteItems`, between `TransactWriteItems` and `DeleteMessage`, and after `DeleteMessage`. Each of those should also be safe, and a mature crash-test harness would verify all of them.

The bug that taught me to separate "correct" from "visibly correct"

One obstacle we hit during the scaling experiment produced a transient balance mismatch of about $89. My first instinct was a hot panic — this was the specific bug the entire semester had been warning us about. I spent a while diagnosing before we realized the issue was not in the transfer code; it was in `verify_balances.py`, which I owned. The verifier used DynamoDB's default eventually-consistent `Scan`, so during active writes it could see one account's new balance on page 1 and the same account's stale balance on page 2 of the scan. The invariant was holding. The observer was broken.

The fix was small — add `ConsistentRead=True` to the Scan parameters, and wait for the SQS queue to fully drain before verifying. But the takeaway was large: **the verification tool must be as carefully designed as the thing it verifies**, or you can't tell the difference between a system bug and a harness bug. In a financial context that distinction is the whole game. Next time I will treat the verifier as first-class code from the start: strongly consistent reads, explicit drain gates, and error messages that distinguish "mismatch" from "still in flight."

This also connected to my midterm observation about observability being about decision quality. The verifier is a piece of observability infrastructure. If your observability lies to you, every decision you make on top of it is wrong. I had framed that idea abstractly in the midterm; I lived it in the final project.

Course concepts that only clicked once I used them

Two-phase commit vs. single-coordinator atomic transactions. The course spent real time on 2PC and its limitations, and I quoted it in my midterm without really having felt it. Implementing the atomic commit in our project — one `TransactWriteItems` that updates sender, receiver, and transaction status in one operation on one service — was the concrete version of "avoid distributed commits when a single-coordinator atomic commit is available." DynamoDB's transactional API is the coordinator here; we pay a small cost (all items must be in the same region) to get atomicity without 2PC's blocking behavior. After the crash test, I understood why that tradeoff is worth it.

Contract-first design. In my midterm I said contract-first improves service boundaries. In the final project I watched it make my life easier every single day: the API handler Pranay wrote and the DynamoDB primitives I wrote communicated through a narrow, typed Go interface, and neither of us had to wait for the other to finish to make progress on our own module. When we had to add the pessimistic path, we added it behind the same interface — the API code did not change at all. That is the Parnas principle working in practice.

Partial failure is the normal case. My HW6 resilience test was a single planned task kill. The final project's Experiment 2 was the same idea extended to a case where state mattered. The general lesson: design assuming that any intermediate step may not finish, and every intermediate step must be safe to abandon. That reframes how you write every block of code in a distributed system — from "what should this do" to "what happens if this never returns."

What I would do differently next time

1. **Write the invariant down first, in one sentence, and pin it somewhere visible.** Our invariant ("no balance change except by a transfer, every transfer atomic") drove every design decision; if I had printed it at the top of every code review comment, we would have caught edge cases earlier.
2. **Build the verification script with the service, not after.** I should have written `verify_balances.py` with `ConsistentRead=True`, the drain wait, and the error-classification logic in Week 1. Instead those only landed when the scaling experiment exposed the harness bug in Week 6.
3. **Crash-test earlier and more places.** Experiment 2 was in the plan for Weeks 3–4, but I could have been running an unplanned crash test against my own worker during pessimistic-lock development. It would have caught the original "lock release after commit" bug much sooner.
4. **Treat IAM trust policies as a Week 1 concern.** Our account did not have a `LabRole`; we discovered that only when Terraform started creating resources. The fix (a small bootstrap script) is a ten-minute job if you do it first and an afternoon of confusion if you discover it after infrastructure work has begun.
5. **Document the schema with its invariant, not just its fields.** Future me should read the DynamoDB schema and immediately understand why `version` exists, why `expires_at` exists, and what invariant they each enforce. A one-paragraph comment at the top of the schema file would make that story permanent.

Closing

My midterm framed systems work as a five-step loop: design, deploy, measure, adapt, verify. I still think that's right — but the final project added weight to every one of those words. "Design"

now means designing for invariants, not just features. "Deploy" now means reproducibly, against real cloud infrastructure, under a documented IAM principal. "Measure" means both Locust-side throughput and worker-side counters, correlated across CloudWatch. "Adapt" means the difference between changing one env var (swapping optimistic for pessimistic locking via Terraform) and rewriting the system. "Verify" means a strongly-consistent Scan, a fully-drained queue, and a harness that distinguishes "bug" from "observer in a race."

The single most valuable thing I am taking out of CS6650 is the habit of pausing before every block of code and asking: *what invariant is this protecting, and what mechanism makes that protection hold under partial failure?* I did not have that habit in HW6. I have it now, and I suspect it will define how I approach every backend system I touch from here.